

Nom:

Prénom:

Exercice 1 : Complexité des algorithmes

1. Si un algorithme a une complexité de $O(n)$, que se passe-t-il pour le temps d'exécution si la taille de la liste est multipliée par 10?
Le temps d'exécution est multiplié par 10.
2. Si un algorithme a une complexité de $O(n^2)$, que se passe-t-il pour le temps d'exécution si la taille de la liste est multipliée par 10?
Le temps d'exécution est multiplié par 100.
3. Si un algorithme a une complexité de $O(\log n)$, que se passe-t-il pour le temps d'exécution si la taille de la liste est multipliée par 64?
Le temps d'exécution augmente de 6.

Exercice 2 : On considère une liste triée L.

1. Ecrire un algorithme de recherche dichotomique pour trouver la position d'un élément x dans la liste. Si l'élément n'est pas présent, l'algorithme doit retourner -1.
2. Indiquer quel est le variant de boucle de cet algorithme.
right - left
3. Indiquer quel est l'invariant de boucle de cet algorithme.
L[left:right] est triée et contient x si x est présent
4. Quelle est la complexité de cet algorithme?
 $O(\log n)$
5. Combien de valeurs sont examinées lors d'un appel à `recherche_dichotomique([1, 3, 4, 6, 9, 12, 14, 15], 9)` et quelle est la valeur renvoyée?
6. Même question pour `recherche_dichotomique([1, 3, 4, 6, 9, 12, 14, 15], 7)`.

Exercice 3 : Débogage

1. Le code suivant pour la recherche naïve contient une erreur classique. Laquelle?
2. Corriger le code pour qu'il fonctionne correctement.

</> Code Python

```
1 def recherche(elt, L):
2     for x in L:
3         if x == elt:
4             return True
5     else:
6         return False
```

Exercice 4 : Débogage

Le code suivant présente une erreur, l'identifier et la corriger.

</> Code Python

```
1 i = 10
2 while i > 0:
3     print(i)
```